

FIT3143 Lab 2

Julia Mallard, Nam Quang Tran



Task 1 – Prime Search using Message Passing Interface (Open MPI)

Distribution of work (naive)

- Naive way of distributing work, work is split up into 4 even blocks
- However, higher ranks (who will get blocks of higher values) will have to do more work
- In this example, rank 3 will have to do 2.28x more work, which is not ideal, as Rank 0 has to wait on all to report back the output.
- Rank 3 will be the bottleneck.

Rank	Work done
0	4,944,695
1	8,011,836
2	9,846,491
3	11,297,539

4 processes and $n = 1000000$



Distribution of work (block cyclic round robin)

- Instead of 4 big blocks, cut the work into smaller slices
- This balances the workload, each rank receives a similar mix of small and large numbers, rather than one rank being left with the expensive high end of the range.
- This reduces the worst case difference of work done between two ranks from 2.28x to 1.12x

Rank	Work done
0	8,012,598
1	8,411,874
2	8,703,415
3	8,972,674

4 processes and $n =$
1000000



What could we have used instead?

Why did we use 8 slices per process? Why not more?

- The law of diminishing returns applies here, and the amount the work evenness out reduces as you go up to 16 or 64 slices.

Why use blocks at all? Why not use round robin on the individual numbers, instead of slices?

- If a certain number of processes are used, some processes can end up with numbers that are only multiples of 2, or other numbers, meaning that some processes can essentially find no/very little primes.



What could we have used instead?

Is there any other load balancer than cyclic round robin?

- Yes, there are methods such as folded/reverse pairing and dynamic self-scheduling, however we did not believe that these methods were necessary, as the block cyclic round robin was fast enough for how simple it was, these other methods are unnecessarily complex for little or no speed up.

Could any other methods of prime searching be used?

- The Sieve of Eratosthenes was mentioned in our feedback for Lab 1, however, we already had a prime searching function that used a square root upper bound, so we wanted to reuse it for apples to apples comparison's sake.



Speed up with varying no. of processors

processors	Time (secs)	speedup	efficiency
1	0.4424	1x	100%
2	0.2568	1.72x	86.1%
3	0.1952	2.27x	75.6%
4	0.1616	2.74x	68.4%
6	0.1623	2.73x	45.4%
8	0.1656	2.67x	33.4%
12	0.1739	2.54x	25.4%

When processors used is within 1-4, the speedup is reasonable, staying above $\frac{2}{3}$ efficiency, even if it does drop off as processors approaches 4. This drop off is most likely due to Amdahl's law. However, when the processors used is greater than 4, the speedup actually gets worse. This is because we have to make use of the efficiency cores, which have a much lower clock speed and processing power than the super cores, and the communication and scheduling of these processors causes overhead.

Run on Apple M5 (4 super, 6 efficiency cores)

n=10,000,000



Speed up with varying n

n	Time (secs) 1 procs	Time (secs) 4 procs	Speedup	efficiency
10^5	0.0013 s	0.0029 s	0.44x	11.1%
$3 \cdot 10^5$	0.0044 s	0.0067 s	0.66x	16.6%
10^6	0.0203 s	0.0201 s	1.01x	25.3%
$3 \cdot 10^6$	0.0856 s	0.0458 s	1.87x	46.7%
10^7	0.4302 s	0.1591 s	2.70x	67.6%
$3 \cdot 10^7$	1.9692 s	0.6581 s	2.99x	74.8%
10^8	10.6936 s	3.4018 s	3.14x	78.6%

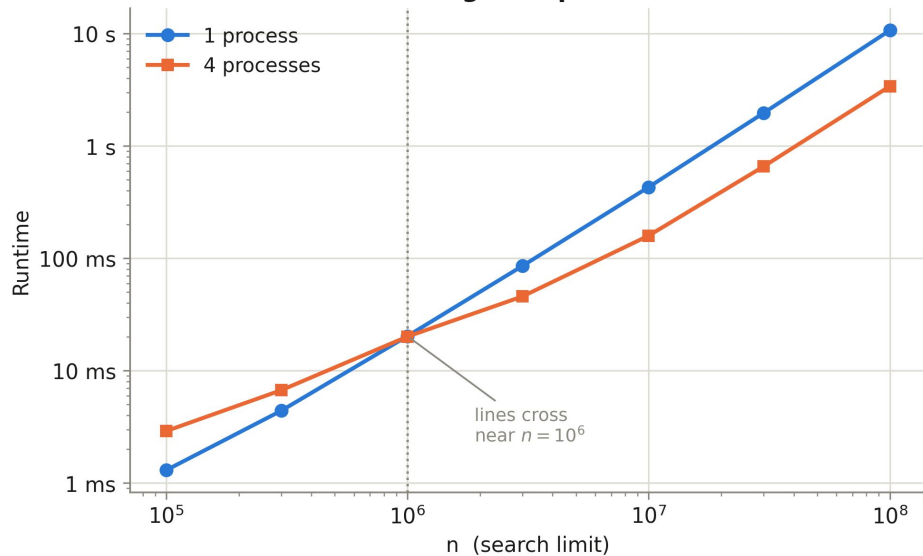
Run on Apple M5 (4 super, 6 efficiency cores)

4 processors

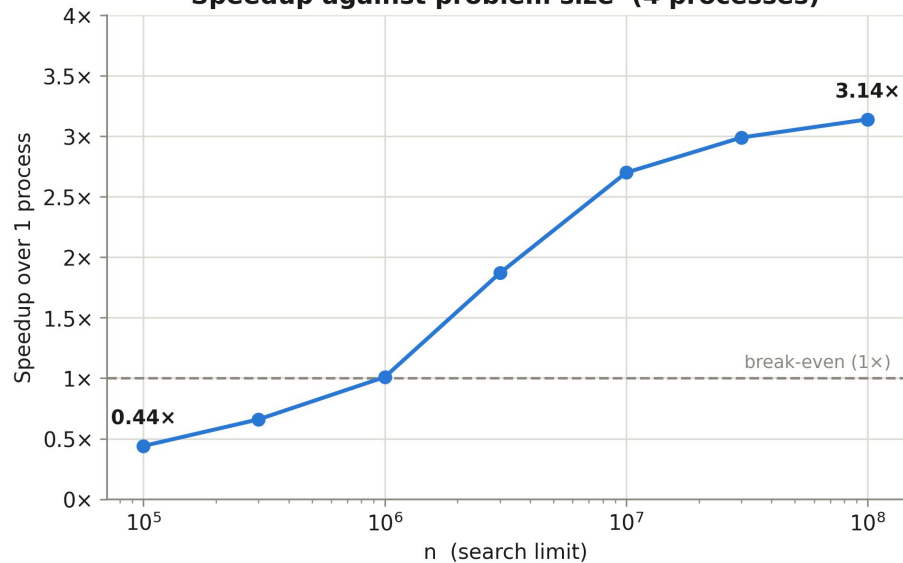
When n is below 10^6 , the implementation is slower than having only 1 processor. This is because of the MPI overhead of having to schedule and run 4 processors at the same time. As n increases, the portion of work dedicated overhead, and actual prime computation flips, causing the speedup to gradually approach the theoretical limit of 4 (as $n \rightarrow \infty$, as n gets higher).



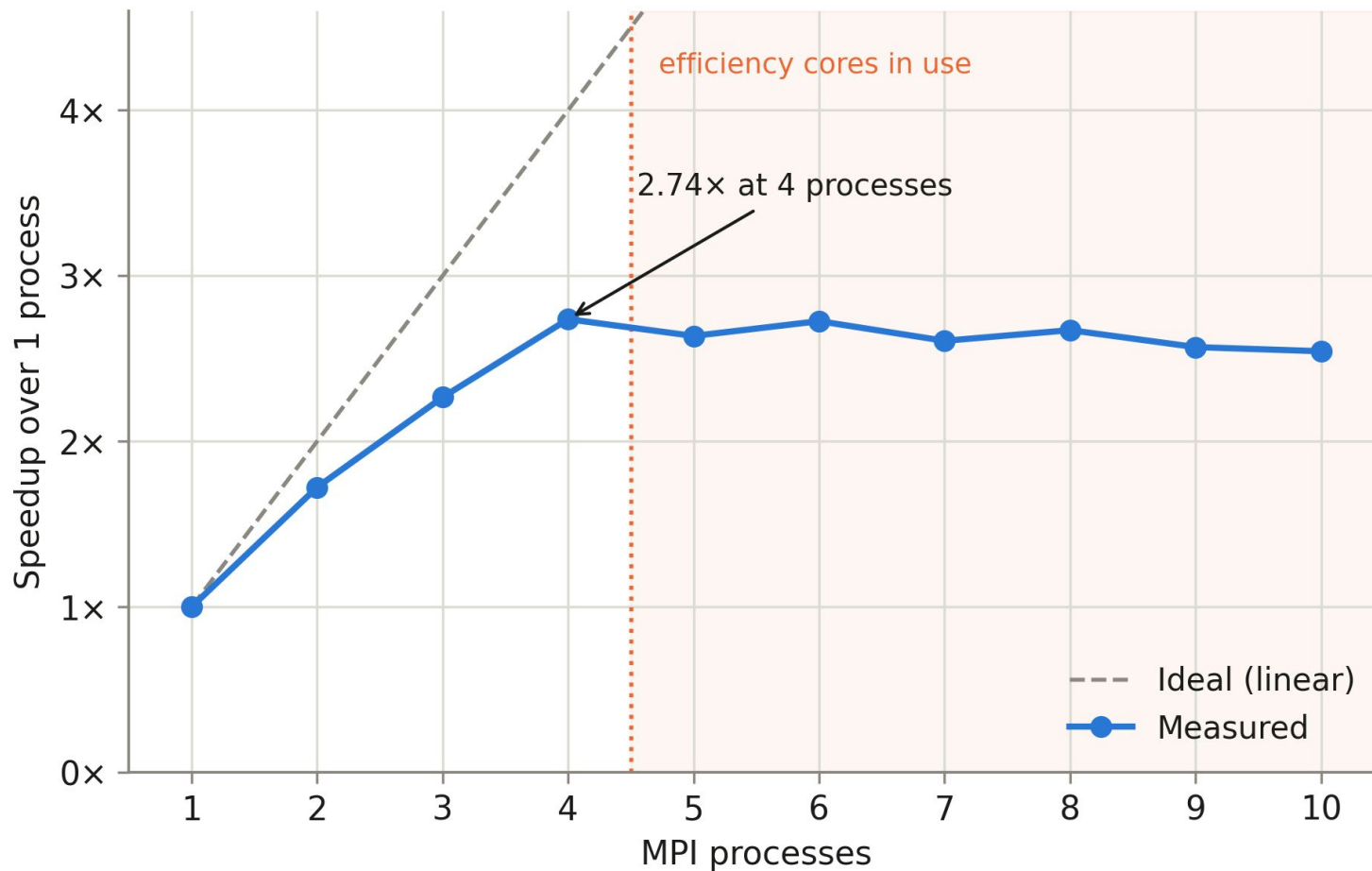
Runtime against problem size



Speedup against problem size (4 processes)

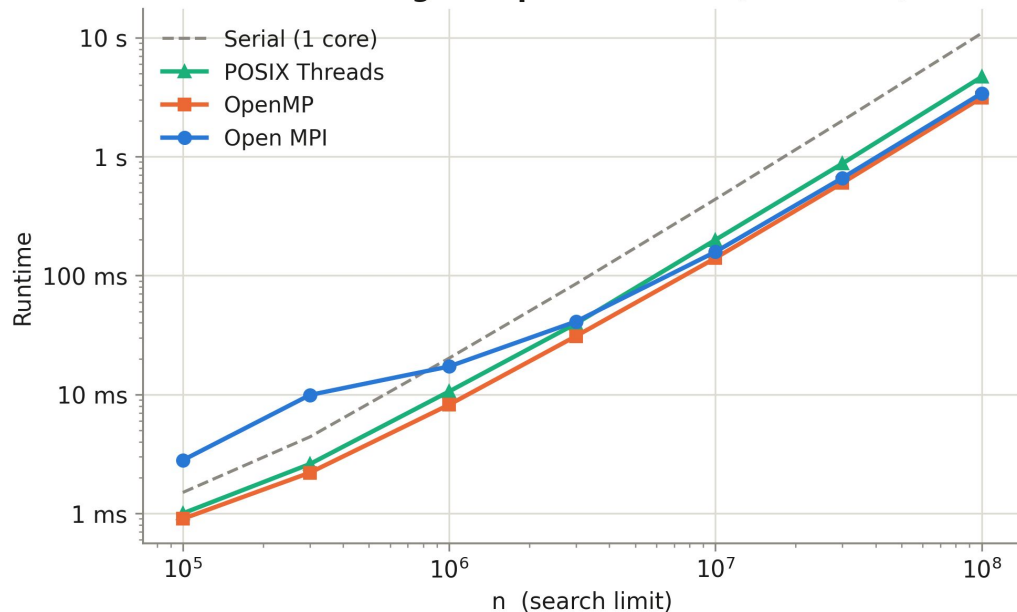


Speedup against process count ($n = 10^7$)



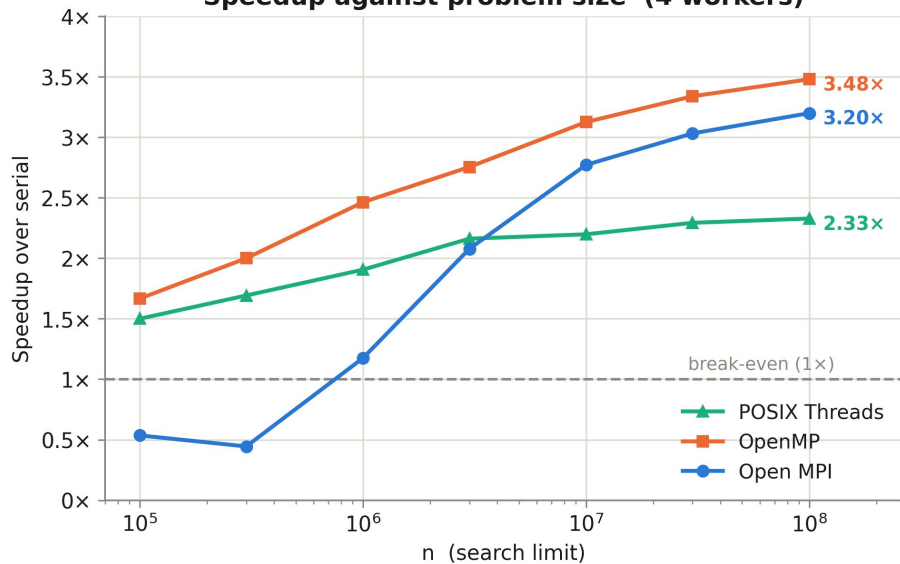
Lab 1 vs Lab 2 comparison graphs

Runtime against problem size (4 workers)

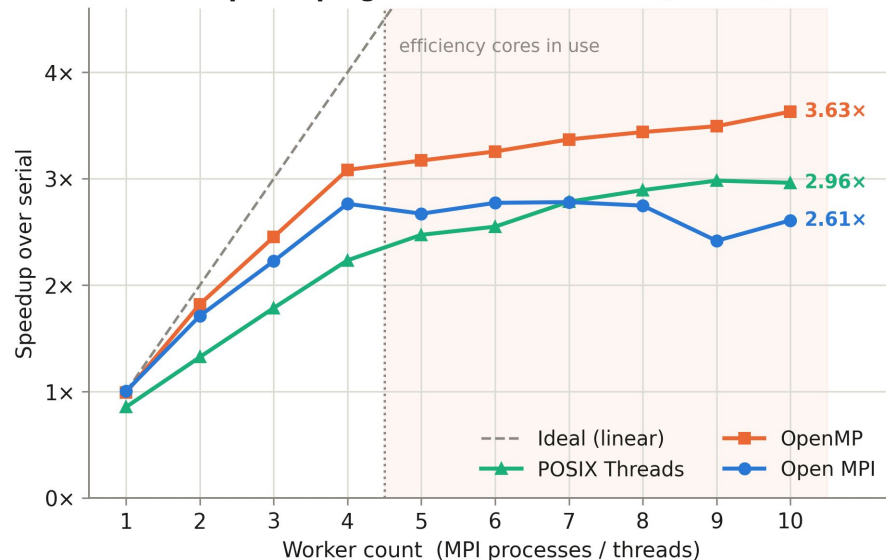


There is significant overhead at lower n for Open MPI (as discussed earlier), but OpenMP and MPI end up converging, and POSIX threads ends up the worst solution, due to the contiguous block splitting (that we “fixed” with slices in MPI).

Speedup against problem size (4 workers)



Speedup against worker count ($n = 10^7$)



OpenMP has the highest speed up, as Open MPI has to gather and sort, which our OpenMP implementation does not need to do.



Task 2 – Prime Search using hybrid OpenMP and Message Passing Interface (Open MPI)

Hybrid approach (OpenMP + MPI)

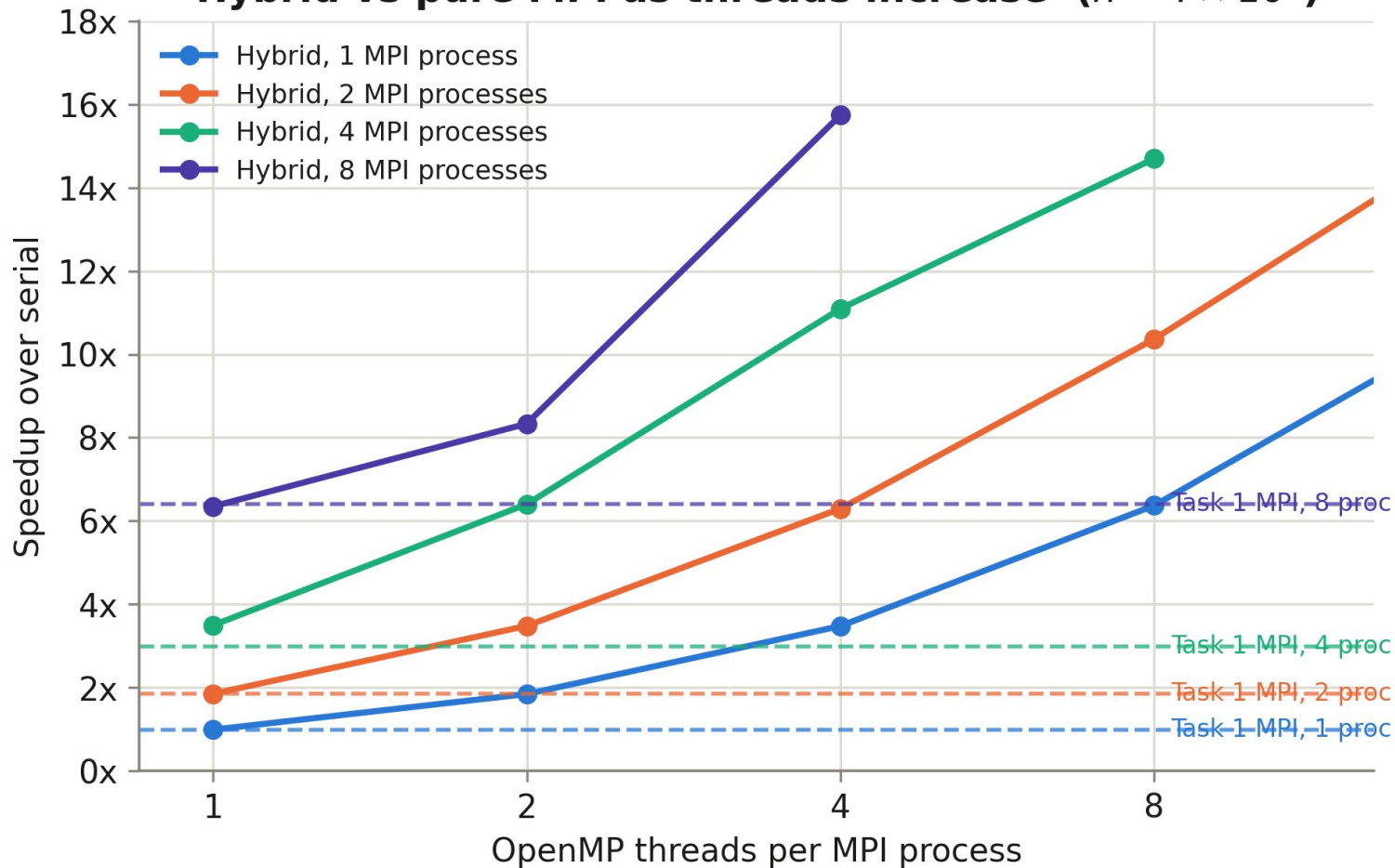
Range $[0, n)$ $\rightarrow$ (processes $\times$ threads $\times$ 8) slices

Work is divided into processes $\times$ threads $\times$ 8 slices, and each thread will get 8 slices on average, and each process will get threads OpenMP threads.

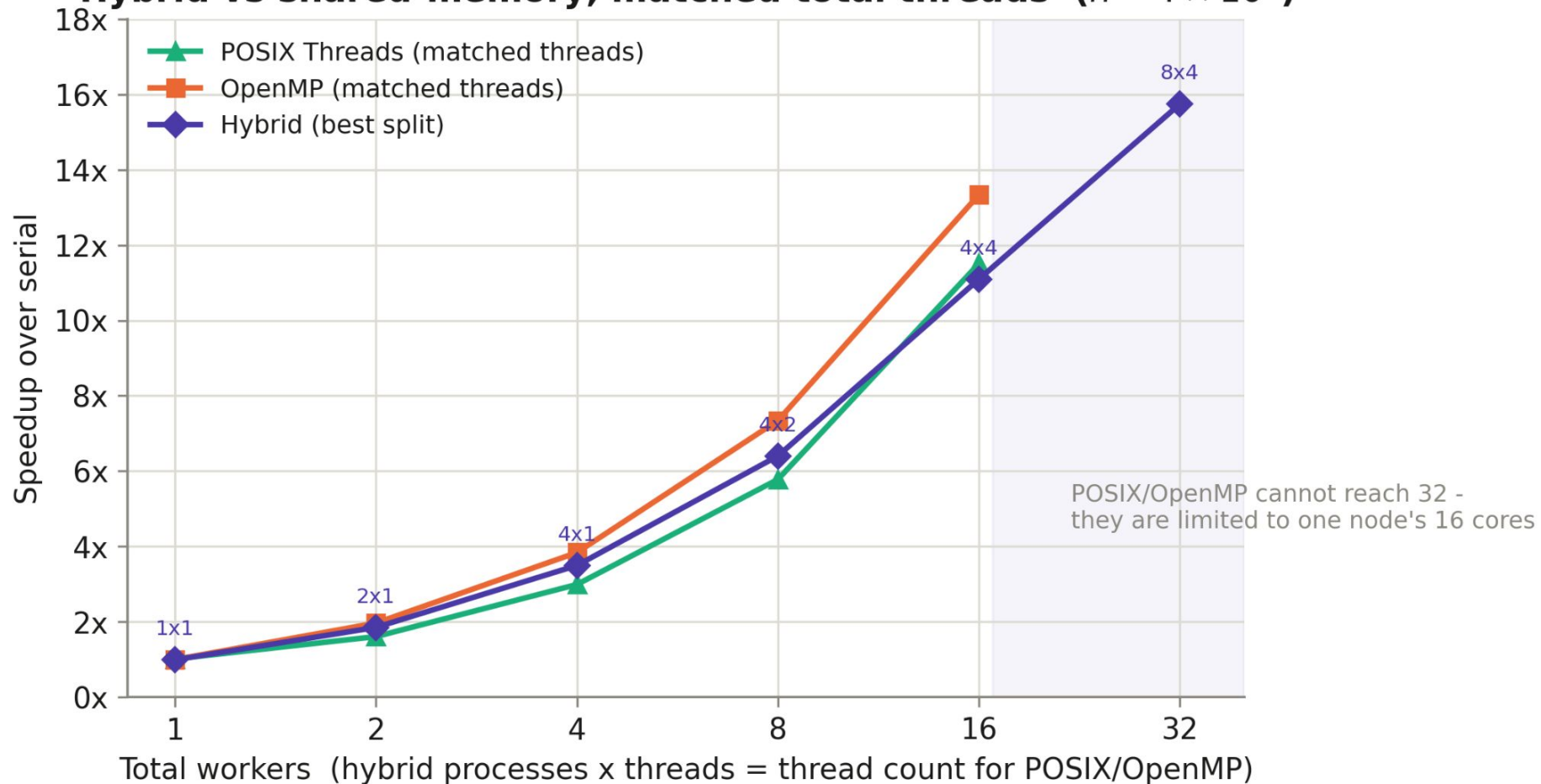
Communication between processes (MPI) using MPI_Bcast and MPI_Gather/MPI_Gatherv, but within a process (OpenMP), there is communication, the work is dynamically scheduled (like in Lab 1).



Hybrid vs pure MPI as threads increase ($n = 4 \times 10^7$)



Hybrid vs shared-memory, matched total threads ($n = 4 \times 10^7$)



What is the speed-up? Is it reasonable?

On CAAS at $n = 4 \times 10^7$, the hybrid reached 11.09 $\times$ on 16 workers (4 processes $\times$ 4 threads) and 15.74 $\times$ on 32 workers (8 $\times$ 4) across two nodes, against a serial baseline of 14.56 s, an efficiency of 69% and 49% respectively. Efficiency falling as workers increase is expected: the gather and the root-side sort and file write do not parallelise, so they occupy a growing share of a shrinking runtime.



How would the speed-up change when you increase & decrease threads, processes, and n?

Increasing either threads or processes raises speedup roughly in proportion to total workers until the core count is reached. At matched worker counts the split is nearly irrelevant below 16 workers: 1×8 , 2×4 , 4×2 and 8×1 differ by under 2%, but above that, thread-heavy splits win: 4×4 (1.31 s) beats 8×2 (1.75 s), because MPI's gather cost grows with the number of ranks while OpenMP's does not. Increasing n improves speedup, as the fixed communication and serial output costs shrink as a proportion of runtime.





Task 3 – Performance evaluation with Amdahl's Law

Task 3: Method & Calculations

Speedup calculations

Empirical:

$$S_{\text{empirical}} = \frac{T_{\text{serial}}}{T_{\text{parallel}}}$$

Amdahl:

$$s = \frac{T_{\text{communication}} + T_{\text{post}}}{T_{\text{total}}} \quad p = 1 - s$$

$$S_{\text{theoretical}} = \frac{1}{s + p/P}$$

P = MPI processes (Task 1)

P = MPI processes × OpenMP threads (Task 2)

Experimental method

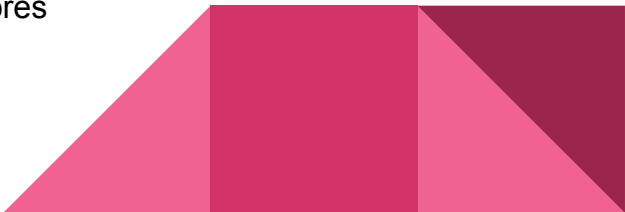
Measured wall-clock time for:

- Serial Week 4 implementation
- Task 1: MPI parallel implementation
- Task 2: Hybrid MPI + OpenMP implementation

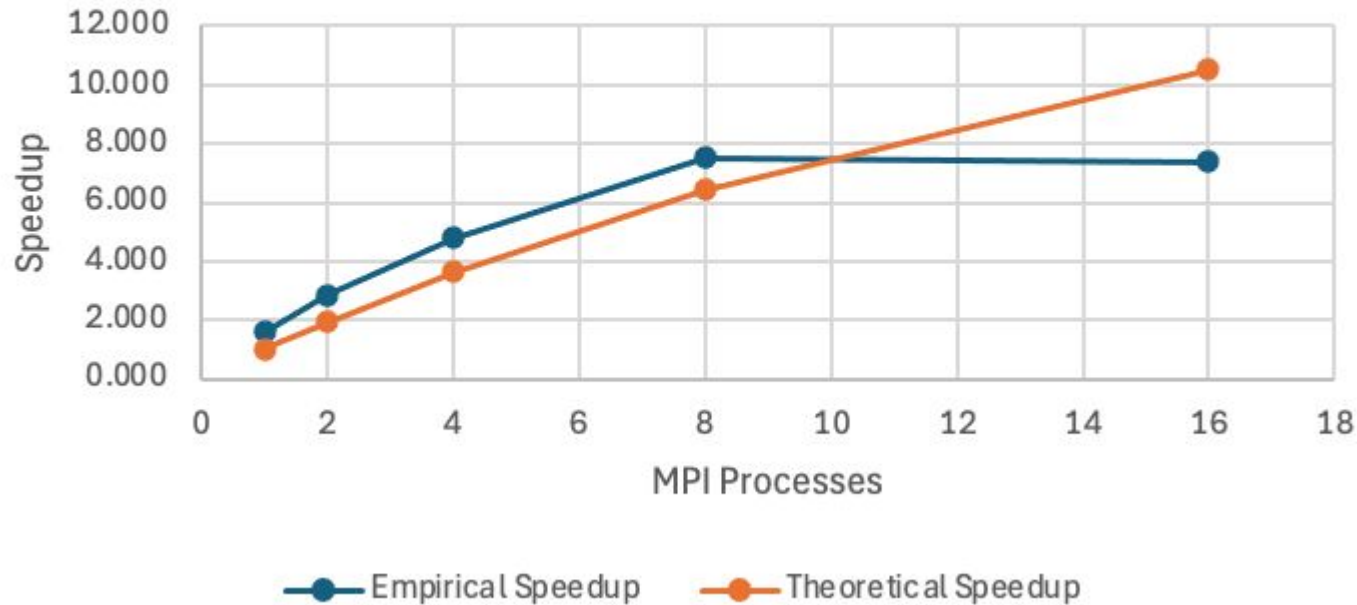
Experiments varied:

- Problem size: **n = 10M** and **50M** shown in results
- MPI processes: **1, 2, 4, 8, 16**
- OpenMP threads: **1, 2, 4, 8, 16**
- 3 runs per configuration, using the average

Hardware: 16 CPU cores



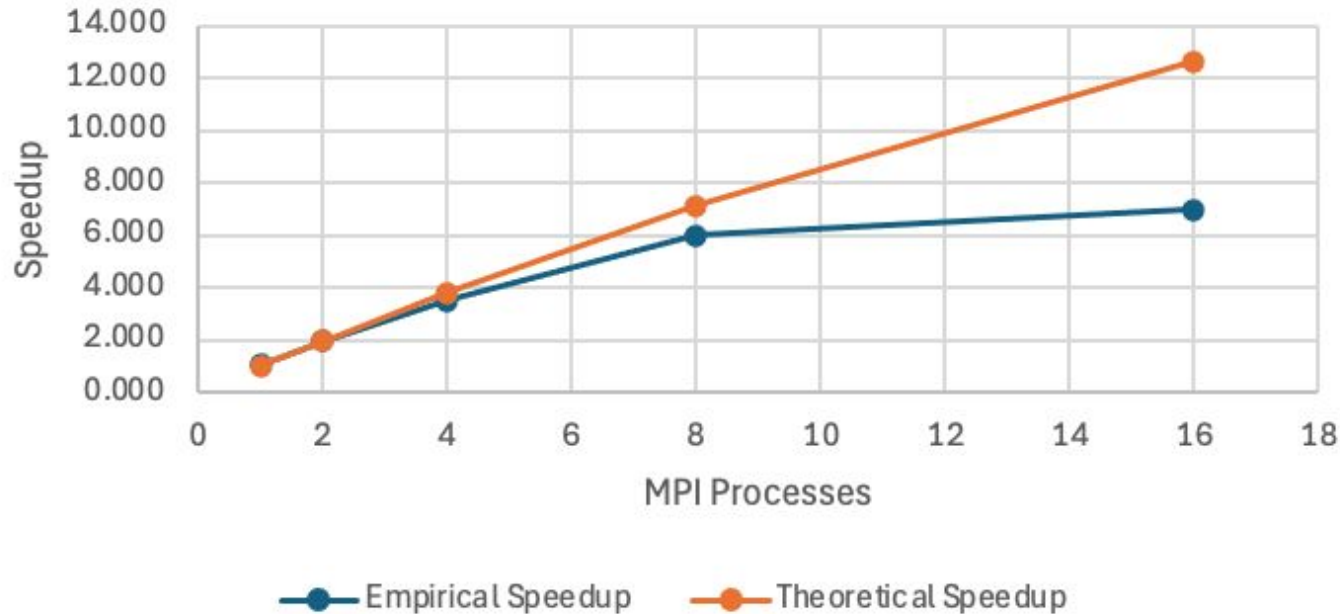
Task 1: Empirical vs Theoretical Speedup with Increasing MPI Processes (n = 10M)



- Empirical speedup increased from **1.59×** to **7.39×** up to 8 processes.
- At 16 processes, empirical speedup slightly decreased to **7.39×**, despite theoretical speedup increasing to **10.52×**.
- The gap at higher process counts indicates the effect of communication and parallelisation overheads.

Experimental environment: Linux Docker environment with an Intel Core i9-9880H @ 2.30 GHz, 16 available CPU cores, Open MPI 4.1.2, and GCC compilation with -O2 optimisation. 3 runs per configuration.

Task 1: Empirical vs Theoretical Speedup with Increasing MPI Processes (n = 50M)



- Empirical speedup increased from **1.06×** to **6.96×** as processes increased to 16.

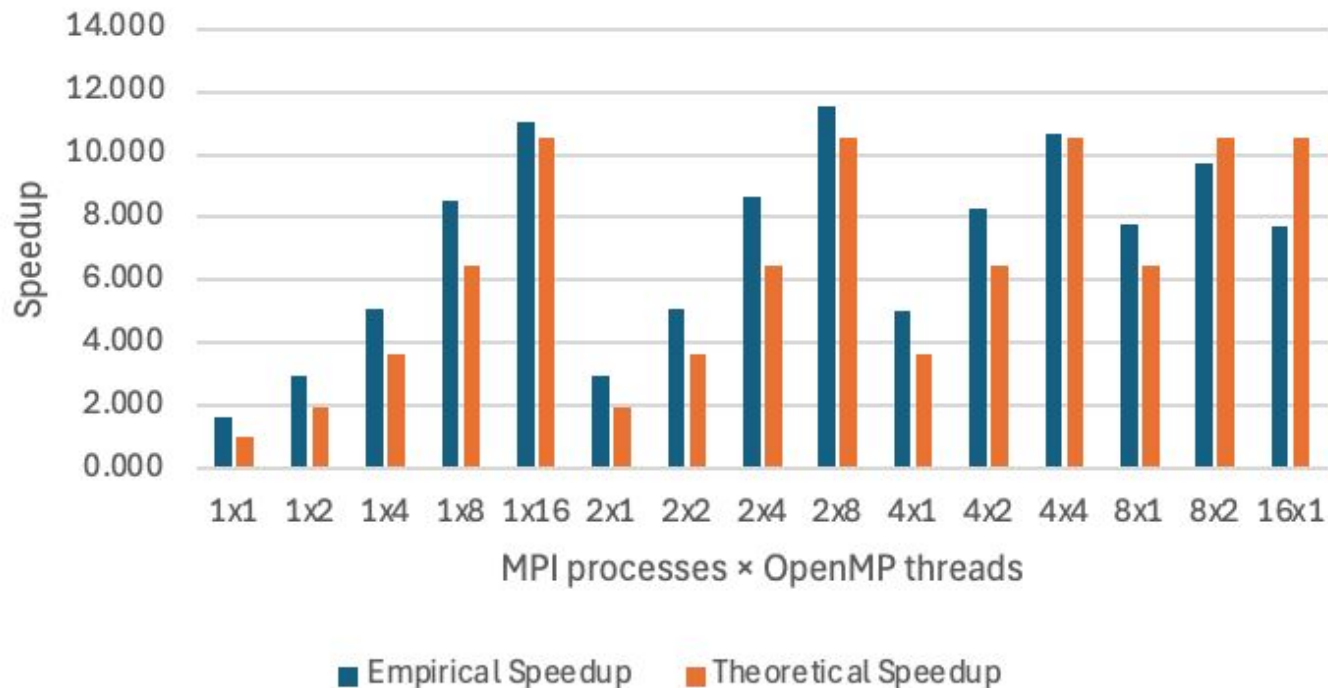
- Theoretical speedup increased from **1.00×** to **12.63×**.

- Gap between empirical and theoretical speedup became larger as no. of processes increased.

- Larger workload → better scaling, but overhead still limits real speedup.

Experimental environment: Linux Docker environment with an Intel Core i9-9880H @ 2.30 GHz, 16 available CPU cores, Open MPI 4.1.2, and GCC compilation with -O2 optimisation. 3 runs per configuration.

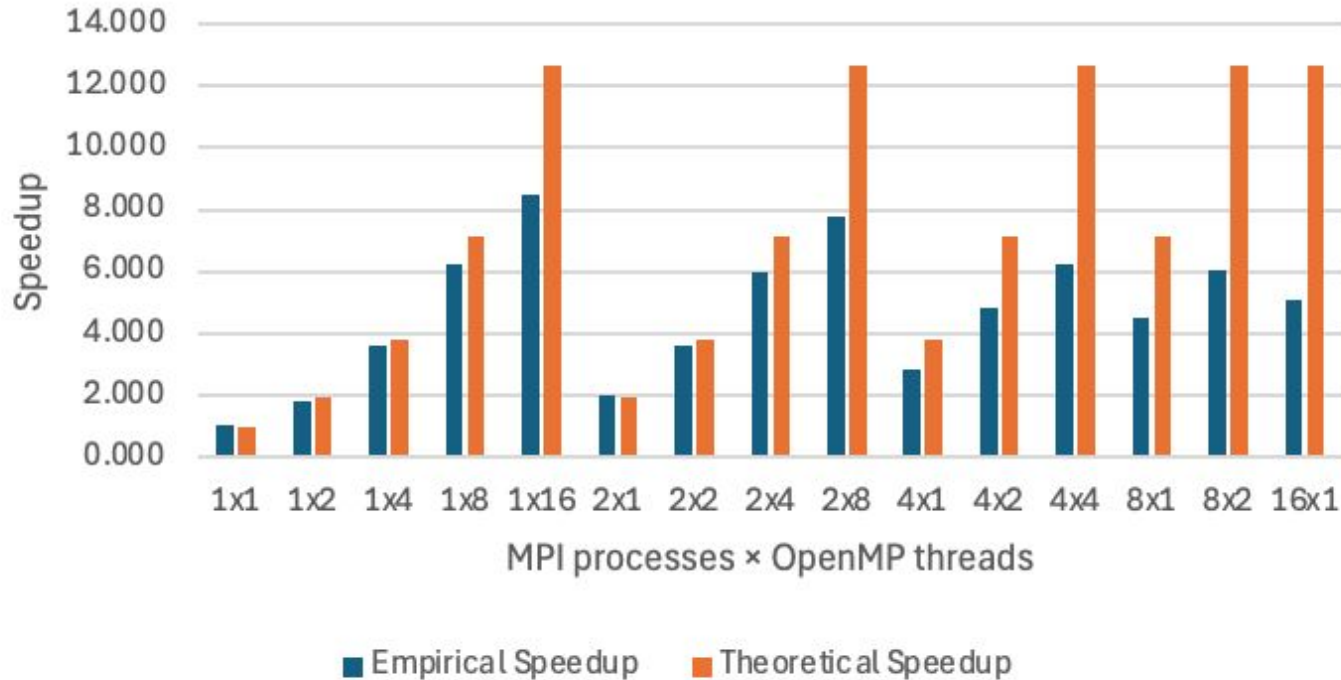
Task 2: Empirical vs Theoretical Speedup (n = 10M)



- Increasing OpenMP threads generally improved empirical speedup for a fixed MPI process count.
- Configurations with the same number of total workers have the same theoretical Amdahl speedup.
- At higher worker counts, empirical speedup increasingly diverged from the theoretical prediction.

Experimental environment: Linux Docker environment with an Intel Core i9-9880H @ 2.30 GHz, 16 available CPU cores, Open MPI 4.1.2, and GCC compilation with -O2 optimisation. 3 runs per configuration.

Task 2: Empirical vs Theoretical Speedup (n = 50M)



- Empirical speedup increased with additional workers, but at a slower rate than theoretical speedup.

- Theoretical speedup reached **12.63×** at 16 workers, while the best empirical speedup was **8.49×**.

- For 16 workers, **1×16** achieved **8.49×**, compared with **16×1** at **5.09×**.

Experimental environment: Linux Docker environment with an Intel Core i9-9880H @ 2.30 GHz, 16 available CPU cores, Open MPI 4.1.2, and GCC compilation with -O2 optimisation. 3 runs per configuration.

Interesting Observations

- More workers do not guarantee proportional speedup.
 - Amdahl's Law predicts an upper bound, not actual performance.
 - Communication, synchronisation, scheduling and other overheads reduce empirical speedup.
 - MPI/OpenMP workload distribution affects performance even with the same total number of workers.
 - Different machines can produce different empirical results due to hardware and system differences.
- 